

Lab assignment: an NLP problem

Applied programming (LIS050)

Robert Östling Amanda Kann
robert@ling.su.se amanda.kann@ling.su.se

Maria Skeppstedt
maria.skeppstedt@ling.su.se

February 4, 2026

1 Purpose and learning outcomes

In this assignment, you will work independently and design your own solution to a problem in natural language processing. This assignment runs over the second half of the course, with some fixed deadlines for handing in a project plan and present your progress during a peer review session.

To pass this assignment (without distinction), you will need to:

- Provide an implementation to one of the NLP problems described below.
- Use a good program structure, i.e.:
 - Divide the code into methods and/or functions that makes it easy to understand, use, and to extend with further functionality
 - Minimal code repetition
- Provide enough documentation to make the code possible to understand for another Python programmer.
- Do **one** of the following for solving the task:
 1. Implement an algorithm relevant for NLP from scratch. (I.e., if you, for instance, choose the Brill’s task. You are supposed to implement the method. It is not enough to use an external library where the method is already implemented.)
 2. Use the scikit-learn library
 3. Use the pyTorch library

- Perform a basic evaluation of your program. (E.g., against an existing gold standard, or a user test with a peer student.)
- Provide a motivation for your choice of method.
- Provide advice to your peers on their projects.

To pass with distinction, you will additionally need to:

- Use a more advanced solution (and also argue for why the solution is more advanced), or implement **two** alternative solutions for solving the task, where you use **two** of the following:
 1. Implement an algorithm relevant for NLP from scratch.
 2. Use the scikit-learn library
 3. Use the pyTorch library
- Use the pydoc tool to automatically generate documentation based on your docstrings.
- Provide an analysis of time and space complexity for your program.

Note that you do *not* need to use state-of-the-art methods, or even a method that performs well. You just need to be able to argue why you hypothesized the method you chose would be suitable. It is totally okay if your hypothesis was wrong. Also note that for this course, you are not meant to use any pre-trained language models more advanced than the static word-vectors introduced in lab 2.

For some problems (e.g. part-of-speech tagging, and text classification), you can measure the performance against a gold standard. For, e.g., the topic extraction and the dialogue systems, you can instead evaluate them by letting human evaluators (e.g. a peer in the course, or someone else) run the program and provide a review of the performance¹.

2 Procedure

There are four deadlines to keep in mind:

- Have a short preliminary project plan ready to present to your peer at the second peer review session.
- To upload the preliminary plan to Athena, describing:
 - Which problem you intend to work on (e.g., “German-to-English machine translation”)

¹See, e.g., the approach for this task <https://sig-edu.org/sharedtask/2023>

- Which type of method you will use (e.g., “statistical machine translation”).
 - Which data set, and where to find it (e.g., chat-task-2024-data, github.com/WMT-Chat-task/chat-task-2024-data/blob/main/train/ende.csv)
 - Which evaluation method (e.g. using the sacreBLEU tool.)
 - Why do you think your choice of method is suitable?
- Present a code draft at the third peer review session.
 - At the last day of the course, upload your code on Athena and present your solution to the class.

3 Problems

3.1 Machine translation

This is *the* original problem in natural language processing, and countless methods have been proposed since the 1950s. Currently, the best systems are based on Transformer models, but for language pairs with less access to training data, rule-based systems or those based on n-gram language models (Statistical Machine Translation, SMT) can still be competitive.

You can use data from the Conference on Machine Translation² (WMT), and the sacreBLEU tool³ for evaluating your translations. Note: do not download the enormous training data files from WMT onto our server. You will not have computational resources to use them efficiently, anyway.

3.2 Grammatical Error Correction

The term Grammatical Error Correction refers to a problem that is in fact a bit more general than the name suggests, and can cover all types of text improvement from spelling corrections to text restructuring. This has traditionally been approached using rule-based methods, but more recently neural machine translation and LLM-based approaches have become popular.

You can use the W&I+LOCNESS v2.1 data from the BEA 2019 shared task⁴ for evaluation and (optionally) training. The ERRANT tool can be used to compute evaluation scores.

²<https://www2.statmt.org/wmt24/index.html>

³<https://github.com/mjpost/sacreBLEU>

⁴<https://www.cl.cam.ac.uk/research/nl/bea2019st/>

3.3 Part-of-speech tagging

This is a classical problem in natural language processing, and is often mentioned as a “solved problem,” at least for languages with sufficient training data. A wide range of methods have been used, and you may have opted to implement a HMM-based part-of-speech tagger in the first assignment of this course. There are many other methods that have been successfully used: Constraint Grammar (Karlsson, 1990), Brill’s rule learning method (Brill, 1992), perceptrons (Östling, 2018), and deep neural networks.

Use the Universal Dependencies treebanks version 2.15⁵ for training and evaluation. If your tagger is fast enough, you should be able to evaluate its performance on all UD languages. Otherwise, at least provide an evaluation for the English-EWT treebank.

3.4 Supervised text classification (and/or unsupervised topic extraction)

There are many applications of text classification. Detecting which texts are spam messages is one very classic and widely used text classification task. Here is a dataset with spam and ham (= non-spam) text messages: <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>. The data set is described by Almeida et al. (2011). Use precision, recall and F-score as evaluation metrics. You need to divide the set into training and test data yourself.

Another options with this data set, could be find out what are the frequent topics among the spam messages. For this, there are no pre-defined classes that decide what is a topic. To find out what are the frequent topics, you can carry out topic modelling or clustering of the dataset. As there are no pre-defined, annotated classes here, the evaluation needs to be carried out by letting human evaluators judge whether relevant and coherent topics/clusters have been extracted.

3.5 Text-based dialogue system

The text-based dialogue system is also a very classic NLP task, from Eliza in the 1960’s to the LLM-based chatbots of today. A dialogue system task could consist of improving how the creatures in the Random Creature game interact. I.e., before eating each other, they could have a short dialogue.

One approach is to use (perhaps slightly modified) entire lines from existing dialogues (Banchs and Li, 2012), and select an appropriate line based on similarity between sentence-vectors. (But make sure not to just repeat the content of the training data, then it is not an NLP problem anymore.)

⁵<http://hdl.handle.net/11234/1-5787>

You could also investigate how well an n-gram model would perform on the task of generating text, or train a machine learning model to predict text.

As training data for a model, you can, for instance, use a subset of the movie dialogue corpus found here: <https://convokit.cornell.edu/documentation/movie.html>.⁶ (Danescu-Niculescu-Mizil and Lee, 2011). In `/home/dsv/mariask/courses/lis050/lab4/movies`, you find code for extracting dialogues for the data.

As evaluation, you could perform some version of the Turing test. I.e., you can investigate whether a human distinguish real movie dialogue from movie dialogue that you have produced.

3.6 Choose your own shared task

Find an old shared NLP task, and solve this task (or a subset of the task). The task must of course require programming (the demands for passing, with or without distinction are still the same). Before starting, make sure the data for the task is still available. Use the evaluation methods/scripts provided by the task organizers.⁷

If you choose this option, you will need to provide a more detailed project plan, than for the other options.

References

- Almeida, T. A., Hidalgo, J. M. G., and Yamakami, A. (2011). Contributions to the study of sms spam filtering: new collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering, DocEng '11*, page 259–262, New York, NY, USA. Association for Computing Machinery.
- Banchs, R. E. and Li, H. (2012). Iris: a chat-oriented dialogue system based on the vector space model. In *Proceedings of the ACL 2012 System Demonstrations, ACL '12*, page 37–42, USA. Association for Computational Linguistics.
- Brill, E. (1992). A simple rule-based part of speech tagger. In *Third Conference on Applied Natural Language Processing*, pages 152–155, Trento, Italy. Association for Computational Linguistics.
- Danescu-Niculescu-Mizil, C. and Lee, L. (2011). Chameleons in imagined conversations: A new approach to understanding coordination of linguistic

⁶Or to download the data, go to <https://www.cs.cornell.edu/~cristian/research#paper-chameleons-in-imagined> and choose “Legacy Data”. Note that the lines in `movie_lines.txt` are not displayed in the correct order. The conversations must instead be constructed by using the ID:s in ‘`move_conversations.txt`’

⁷Here are some examples of NLP and other tasks: <https://www.codabench.org>

- style in dialogs. In *Proceedings of the Workshop on Cognitive Modeling and Computational Linguistics, ACL 2011*.
- Karlsson, F. (1990). Constraint grammar as a framework for parsing running text. In *COLING 1990 Volume 3: Papers presented to the 13th International Conference on Computational Linguistics*.
- Östling, R. (2018). Part of speech tagging: Shallow or deep learning? *North European Journal of Language Technology*, (5):1–15.